Walk Attention: Path Algebras of Quivers as a Sparse Multi-Hop Attention for Mitigating Over-Squashing in GNNs

Carlos Isaac Zainea Maya[1] and Agustín Moreno Cañadas[1]

Universidad Nacional de Colombia, Departamento de Matemáticas, Bogotá, Colombia
{cizaineam,amorenoca}@unal.edu.co

Abstract. Message-passing graph neural networks (GNNs) suffer from *over-squashing*: when information must travel along many long paths into a node, it is compressed into a fixed-width vector and the signal is lost. We introduce **Walk Attention**, an architecture that addresses over-squashing through the *path algebra* $\Bbbk Q$ of the directed graph (quiver) underlying the data. The graded components of $\Bbbk Q$ count directed walks of a given length between two vertices and thus give an exact, algebraic description of the multi-hop reachability structure of the network. We use this structure to define a sparse, multi-hop attention: a node attends directly to every vertex it can reach by a walk of length g, with the path algebra supplying the attention *support* and the network learning the attention *weights*. We give a self-contained construction of quivers, their path algebras, and the walk operators, and we relate them to message passing and to Transformer attention. On a controlled family of bottleneck graphs with tunable over-squashing, Walk Attention reaches 1.000 ± 0.000 accuracy over five seeds at problem depths where a one-hop graph-attention network collapses to chance (0.26–0.44) and a fixed-weight multi-hop aggregator reaches only 0.50–0.62. We additionally report a negative result that motivates the design: using the path-algebra *quotient* to *collapse* equivalent walks discards the multiplicity that the task needs and fails. The path algebra is therefore valuable not as a quotient that compresses information, but as the object that *defines the attention support*.

Keywords: Graph neural networks · Over-squashing · Path algebra · Quiver · Attention · Message passing.

1 Introduction

Graph neural networks (GNNs) built on the message-passing paradigm [4] update each node by aggregating messages from its immediate neighbours and iterating this local rule across layers. After k layers, information from nodes k hops away reaches a target node. While simple and effective, this scheme has a well-documented failure mode: *over-squashing* [1]. When the number of

paths that funnel information into a node grows—typically exponentially with distance—the messages carried by all of those paths must be compressed into a single fixed-width vector. The receiver cannot disentangle them, and long-range signal is lost. Over-squashing is the principal obstacle to learning tasks whose labels depend on distant or globally-distributed information.

Existing remedies act on the graph or the aggregation: *rewiring* adds or reweights edges to widen bottlenecks [8], graph *coarsening* and *pooling* reduce the graph while trying to preserve structure, and curvature-based analyses explain *where* squashing occurs [3]. A complementary line of work replaces local aggregation by global *attention* as in Transformers [9], at the cost of an all-pairs interaction that ignores graph structure.

In this paper we take a different route, grounded in the representation theory of quivers. A *quiver* is a directed graph, possibly with multiple arrows [7,2], and its *path algebra* $\Bbbk Q$ has the directed walks of the quiver as a basis, with concatenation as multiplication. The graded piece $e_i \cdot \Bbbk Q_g \cdot e_j$ of this algebra has a basis consisting of the directed walks of length g from vertex i to vertex j; its dimension equals $(A^g)_{ij}$, the corresponding entry of the g-th power of the adjacency matrix. The path algebra therefore provides an *exact, algebraic description of multi-hop reachability and multiplicity.*

Our central observation is that this algebraic structure is precisely what is needed to define a principled, sparse, multi-hop attention. We let each node attend directly to every vertex reachable from it by a walk of length g: the path algebra supplies the *support* of the attention (which pairs may interact at range g), and the network learns the *weights* (how much). Because the walk-reachability support is sparse—typically a small fraction of all node pairs—this is far cheaper than full self-attention, while still acting at a distance and thereby bypassing the iterative bottleneck that causes over-squashing. We call the resulting architecture **Walk Attention**.

Contributions.

1. We give a *self-contained* construction (Section 3) of quivers, path algebras, graded components, and the associated *walk operators*, aimed at a pattern-recognition audience and independent of prior exposure to representation theory.
2. We define **Walk Attention** (Section 4): a learned multi-hop attention whose support is the path-algebra walk-reachability structure, and we relate it to message passing and to Transformer attention.
3. On a controlled bottleneck family with tunable over-squashing (Section 5), Walk Attention attains 1.000 ± 0.000 accuracy over five seeds where one-hop graph attention collapses to chance and a fixed-weight multi-hop aggregator only partially succeeds.
4. We report a *negative* result that sharpens the design: collapsing equivalent walks via the path-algebra *quotient* discards needed multiplicity and fails. The quotient's role is to *shape* the support, not to compress the signal.

2 Related Work

Over-squashing. Alon and Yahav [1] identified over-squashing and proposed the NeighborsMatch probe, on which standard GNNs degrade sharply with the problem radius. Topping et al. [8] connected over-squashing to negative graph curvature and proposed stochastic discrete Ricci flow (SDRF) rewiring; Di Giovanni et al. [3] gave a sensitivity analysis relating squashing to width, depth, and topology. These works either modify the graph or characterise the phenomenon. We instead change the aggregation operator, attending at a distance over an algebraically-defined support.

Attention and graph Transformers. Graph attention networks [10] learn one-hop attention weights; full graph Transformers apply Transformer self-attention [9] over all node pairs, recovering long range at quadratic cost and with positional/structural encodings to reinject topology. Walk Attention sits between these: it is attention, but its support is the sparse multi-hop walk-reachability of the quiver rather than one hop or all pairs.

Path and walk structure. Path-counting and powers of the adjacency matrix are classical, and the path algebra of a quiver is a standard object in representation theory [7,2]. Brauer-configuration algebras and related path-algebraic constructions have recently been applied to combinatorial and applied problems by Moreno Cañadas and collaborators [6,5]. To our knowledge, the use of the graded path algebra to define the *support* of a learned multi-hop graph attention is new.

3 The Path Algebra of a Quiver

This section is self-contained. We build, from first principles, the algebraic objects that the model in Section 4 relies on: quivers, paths, the path algebra, its grading by length, and the *walk operators* that encode multi-hop reachability. No prior background in representation theory is assumed; standard references are [7,2].

3.1 Quivers and paths

Definition 1 (Quiver). *A* quiver *is a quadruple $Q = (Q_0, Q_1, s, t)$ where Q_0 is a finite set of* vertices*, Q_1 is a finite set of* arrows*, and $s, t\colon Q_1 \to Q_0$ assign to each arrow α its* source *$s(\alpha)$ and* target *$t(\alpha)$. We write $\alpha\colon i \to j$ when $s(\alpha) = i$ and $t(\alpha) = j$.*

A quiver is thus a directed graph in which multiple arrows between the same ordered pair of vertices are allowed (a *multigraph*). For pattern recognition this is exactly the data of a directed relational structure: vertices are entities and arrows are directed relations, with parallel arrows recording multiplicity. We write $n = |Q_0|$ and identify Q_0 with $\{1, \ldots, n\}$.

Definition 2 (Path). *A* path of length $g \geq 1$ *in* Q *is a sequence of arrows* $p = \alpha_g \cdots \alpha_2 \alpha_1$ *such that* $t(\alpha_r) = s(\alpha_{r+1})$ *for* $1 \leq r < g$*; that is, the arrows compose head-to-tail. Its source is* $s(p) = s(\alpha_1)$ *and its target is* $t(p) = t(\alpha_g)$*. For each vertex* i *we also admit a* trivial path e_i *of length* 0 *with* $s(e_i) = t(e_i) = i$*.*

We read paths right-to-left, as for function composition. A path is a directed *walk*: vertices and arrows may repeat. The number of distinct walks of a given length between two vertices is the quantity that drives over-squashing, and the path algebra makes it algebraically explicit.

3.2 The path algebra and its grading

Let $\Bbbk$ be a field (in practice $\Bbbk = \mathbb{R}$).

Definition 3 (Path algebra). *The* path algebra $\Bbbk Q$ *is the* $\Bbbk$*-vector space with basis the set of all paths of* Q *(including the trivial paths* e_i*), endowed with the bilinear multiplication given on basis paths by concatenation:*

$$q \cdot p \;=\; \begin{cases} q\,p & \text{if } s(q) = t(p), \\ 0 & \text{otherwise,} \end{cases}$$

and extended linearly. The trivial paths satisfy $e_{t(p)}\,p = p = p\,e_{s(p)}$ *and* $e_i e_j = \delta_{ij} e_i$*, so* $\{e_i\}_{i \in Q_0}$ *is a complete set of orthogonal idempotents and* $1 = \sum_i e_i$ *is the unit.*

The multiplication respects path length: concatenating a path of length g with one of length h yields a path of length $g+h$ (or zero). Hence $\Bbbk Q$ is a *graded* algebra,

$$\Bbbk Q \;=\; \bigoplus_{g \geq 0} \Bbbk Q_g, \qquad \Bbbk Q_g \cdot \Bbbk Q_h \subseteq \Bbbk Q_{g+h},$$

where $\Bbbk Q_g$ is the subspace spanned by the paths of length exactly g. In particular $\Bbbk Q_0 = \bigoplus_i \Bbbk e_i$ and $\Bbbk Q_1$ is spanned by the arrows.

The idempotents e_i let us isolate the paths between a fixed pair of vertices. For $i, j \in Q_0$ and $g \geq 0$, the subspace

$$e_j \cdot \Bbbk Q_g \cdot e_i \;\subseteq\; \Bbbk Q$$

is spanned exactly by the paths of length g from i to j. Its dimension is therefore the *number of directed walks* of length g from i to j. The following elementary proposition makes the link to linear algebra precise and is the computational backbone of the model.

Proposition 1 (Graded multiplicity equals walk count). *Let* $A \in \mathbb{N}^{n \times n}$ *be the adjacency matrix of* Q*,* $A_{ij} = \#\{\alpha \in Q_1 : \alpha \colon i \to j\}$*. Then for all* $g \geq 0$ *and all* $i, j \in Q_0$*,*

$$\dim_{\Bbbk}\big(e_j \cdot \Bbbk Q_g \cdot e_i\big) \;=\; (A^{\,g})_{ij}.$$

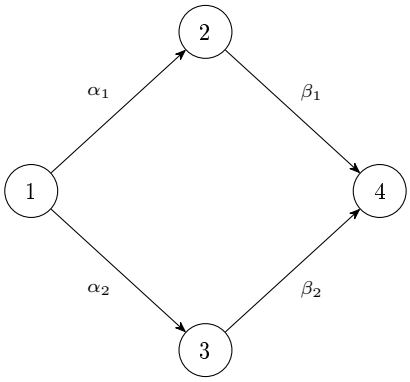

Fig. 1. The diamond quiver of Example 1. The two length-2 walks $1 \rightsquigarrow 4$ are parallel; $\dim(e_4 \Bbbk Q_2 e_1) = 2$.

Proof. By induction on g. For $g = 0$, $e_j \Bbbk Q_0 e_i = \delta_{ij}\, \Bbbk e_i$, of dimension $\delta_{ij} = (A^0)_{ij} = (\mathrm{Id})_{ij}$. For $g = 1$, a basis of $e_j \Bbbk Q_1 e_i$ is the set of arrows $i \to j$, of cardinality A_{ij}. Assume the claim for $g - 1$. Every path of length g from i to j factors uniquely as $\alpha \cdot p$, where p is a path of length $g - 1$ from i to some intermediate vertex r and $\alpha \colon r \to j$ is an arrow. The number of such factorisations is $\sum_r A_{rj}\,(A^{g-1})_{ir} = (A^g)_{ij}$, which proves the claim. □

We call the integer $n_g(i, j) := (A^g)_{ij} = \dim_{\Bbbk}(e_j \Bbbk Q_g e_i)$ the *walk multiplicity* from i to j at range g. It is exactly the number of length-g messages that a message-passing GNN forces through node j from node i after g layers, and hence the source of over-squashing.

3.3 An explicit example

Example 1 (A diamond quiver). Let Q have vertices $\{1, 2, 3, 4\}$ and arrows $\alpha_1 \colon 1 \to 2$, $\alpha_2 \colon 1 \to 3$, $\beta_1 \colon 2 \to 4$, $\beta_2 \colon 3 \to 4$ (Figure 1). Its adjacency matrix and the square are

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad A^2 = \begin{pmatrix} 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

There are two length-2 walks from 1 to 4, namely $\beta_1\alpha_1$ and $\beta_2\alpha_2$, so $\dim(e_4 \Bbbk Q_2 e_1) = (A^2)_{14} = 2$, in agreement with Proposition 1. These two walks are *parallel*: same source, same target, same length. Whether they should be treated as one channel or two is a modelling decision we return to in Section 5.

3.4 Walk operators

Proposition 1 lets us turn the graded path algebra into a family of linear operators on node features, one per range g. These *walk operators* are the bridge to the neural model.

Definition 4 (Walk operator). *For each range $g \geq 1$ define the matrix $W^{(g)} \in \mathbb{R}^{n\times n}$ by*

$$W^{(g)}_{ji} \;=\; \frac{(A^g)_{ij}}{Z_g},$$

where $Z_g = \max_j \sum_i (A^g)_{ij}$ is a single normalising constant for the range. The walk-reachability support *at range g is the index set*

$$\mathcal{S}_g \;=\; \big\{(j,i) : (A^g)_{ij} > 0\big\} \;=\; \big\{(j,i) : \dim(e_j \Bbbk Q_g e_i) > 0\big\}.$$

Acting on a feature matrix $H \in \mathbb{R}^{n\times d}$, the operator $W^{(g)}H$ sends to each node j a weighted sum of the features of every node i that reaches j by a length-g walk, the weight being proportional to the walk multiplicity $n_g(i,j)$. Two facts are worth stressing.

(i) Global normalisation, not row normalisation. We divide by a single constant Z_g per range rather than normalising each row to sum to one. Row normalisation would map the multiplicities to a uniform distribution over the reachable sources and would erase the very multiplicity information that distinguishes $W^{(g)}$ from a plain reachability mask; the global constant keeps the relative multiplicities intact while bounding the magnitude.

(ii) Sparsity. The support $\mathcal{S}_g$ is exactly the nonzero pattern of A^g. On the structured graphs of interest it is a small fraction of all n^2 pairs (see Section 5), so $W^{(g)}$ can be stored and applied in $O(|\mathcal{S}_g|)$ rather than $O(n^2)$.

The fixed-weight operator $W^{(g)}$ already mitigates over-squashing, because it lets node j read from range-g sources *directly*, in one step, instead of forcing the signal through g rounds of local message passing. Its limitation—which motivates Walk Attention—is that the weights are fixed by multiplicity and cannot *select* which reachable source matters. We address this in Section 4 by learning the weights over the same support.

3.5 The quotient: a relation among walks

Representation theory routinely passes from $\Bbbk Q$ to a quotient $\Bbbk Q/I$ by an *admissible ideal* I, identifying paths that are declared equal. In the diamond of Example 1, the relation $\beta_1\alpha_1 \equiv \beta_2\alpha_2$ generates an ideal I for which $\dim\big(e_4(\Bbbk Q/I)_2 e_1\big) = 1$: the two parallel walks collapse to a single class. More generally, $\dim\big(e_j(\Bbbk Q/I)_g e_i\big) \leq n_g(i,j)$, so the quotient yields a structurally smaller multiplicity.

It is tempting to use this quotient directly as an over-squashing remedy: replace the walk multiplicities $n_g(i,j)$ by the smaller quotient dimensions, thereby "de-duplicating" redundant paths before aggregation. We tested this and found it *counterproductive* (Section 5): collapsing parallel walks destroys multiplicity

information that downstream tasks may require. The constructive role of the quotient, then, is not to compress the signal but to *reshape the support*—e.g. to route distinct path classes to distinct attention heads—a direction we leave to future work. In the model that follows we use the path algebra only through the reachability support $\mathcal{S}_g$ and the (unquotiented) walk operators $W^{(g)}$.

4 Walk Attention

We now define the architecture. The guiding principle is the observation that the multi-hop walk operator of Section 3 is structurally an *attention*: it aggregates, for each target node, a weighted combination of the features of the nodes that reach it. Transformer self-attention [9] has the same shape but with all-pairs support and *learned* weights; graph attention [10] restricts the support to one hop. Walk Attention keeps the learned weights of attention and takes the support from the path algebra: the multi-hop, sparse walk-reachability $\mathcal{S}_g$.

4.1 The Walk Attention layer

Fix a range g with reachability support $\mathcal{S}_g$ (Definition 4). A Walk Attention layer with H heads and head dimension C transforms node features $x \in \mathbb{R}^{n \times d_{\text{in}}}$ as follows. For each head h it computes queries, keys and values by linear maps,

$$Q^h = x\,W_Q^h, \qquad K^h = x\,W_K^h, \qquad V^h = x\,W_V^h, \qquad W_\bullet^h \in \mathbb{R}^{d_{\text{in}} \times C},$$

and then attends *only over the reachable pairs*: for every $(j, i) \in \mathcal{S}_g$,

$$\ell_{ji}^h = \frac{1}{\sqrt{C}}\,\langle Q_j^h,\, K_i^h \rangle, \tag{1}$$

$$\alpha_{ji}^h = \operatorname{softmax}_{i:\,(j,i)\in\mathcal{S}_g}\!\left(\ell_{ji}^h\right), \tag{2}$$

$$z_j^h = \sum_{i:\,(j,i)\in\mathcal{S}_g} \alpha_{ji}^h\, V_i^h. \tag{3}$$

The softmax in (2) is taken, for each target j, over exactly the sources i that reach j by a length-g walk—a *segmented* softmax over the support, never over all n nodes. The head outputs are concatenated (or averaged in the final layer) and a residual "root" term is added:

$$\mathrm{WA}_g(x)_j \;=\; \Big\|_{h=1}^{H} z_j^h \;+\; x_j W_{\text{root}}.$$

Because (1)–(3) only ever touch the pairs in $\mathcal{S}_g$, the layer costs $O(|\mathcal{S}_g|\,H\,C)$ time and memory, not $O(n^2)$. The support is precomputed once per graph from Proposition 1 (the nonzero pattern of A^g). Figure 2 summarises the data flow.

Remark 1 (Why learned weights matter). The fixed-weight operator $W^{(g)}$ of Definition 4 is the special case in which α_{ji}^h is replaced by the constant $n_g(i,j)/Z_g$. It reaches every range-g source but weights them by multiplicity alone, so it cannot *select* which source carries the relevant signal. The learned attention (1)–(2) restores this selectivity while keeping the same multi-hop support. Section 5 shows this distinction is decisive.

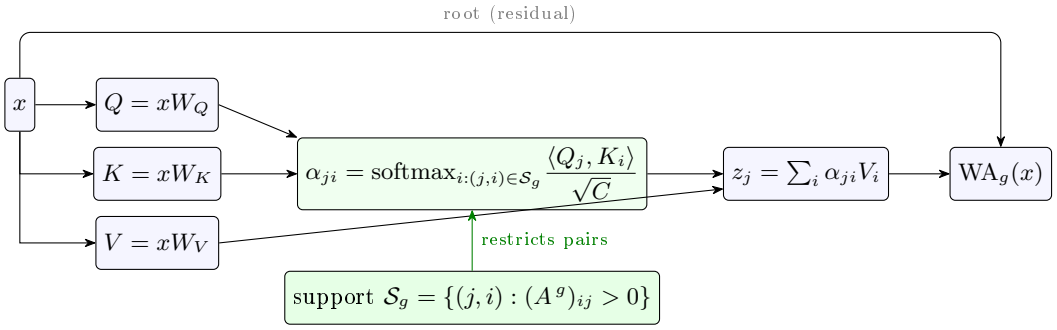

Fig. 2. One Walk Attention layer at range g. Queries, keys and values are linear maps of the node features; attention is computed *only* over the walk-reachable pairs $\mathcal{S}_g$ supplied by the path algebra (the nonzero pattern of A^g), then aggregated and added to a residual root term. The green support box is what distinguishes Walk Attention from dense ($\mathcal{S}_g$ = all pairs) and one-hop ($\mathcal{S}_g$ = edges) attention.

Algorithm 1 Walk Attention layer at range g (sparse).

Require: features $x \in \mathbb{R}^{n\times d_{\text{in}}}$; support $\mathcal{S}_g = \{(j,i) : (A^g)_{ij} > 0\}$
$Q, K, V \leftarrow xW_Q,\ xW_K,\ xW_V$ ▷ per head; shapes $n \times HC$
for each $(j,i) \in \mathcal{S}_g$ and head h **do**
  $\ell^h_{ji} \leftarrow \langle Q^h_j, K^h_i\rangle / \sqrt{C}$
end for
$\alpha^h_{ji} \leftarrow$ segmented-softmax over $\{i : (j,i) \in \mathcal{S}_g\}$
$z^h_j \leftarrow \sum_i \alpha^h_{ji} V^h_i$ ▷ scatter-add into targets
return $\big\|_h z^h + xW_{\text{root}}$

4.2 The network

A depth-L Walk Attention network stacks one layer per range $g = 1, \ldots, L$. Layer g uses the support $\mathcal{S}_g$ derived from A^g, so successive layers read from progressively longer walks. Each layer is followed by layer normalisation, an ELU nonlinearity and dropout; a final multilayer perceptron maps the node embedding to task outputs:

$$x^{(g)} \;=\; \mathrm{dropout}\Big(\mathrm{elu}\big(\mathrm{LN}\big(\mathrm{WA}_g\big(x^{(g-1)}\big)\big)\big)\Big), \qquad \hat{y} \;=\; \mathrm{MLP}\big(x^{(L)}\big).$$

Layer normalisation is important in practice: without it the learned attention is high-variance across initialisations (it reaches the optimum on some seeds and not others); with it, together with a short learning-rate warmup and gradient clipping, training converges reliably (Section 5). Algorithm 1 summarises one layer.

Table 1. Walk Attention against related aggregation operators.

Operator	Support	Weights	Cost
GCN / message passing	1-hop	fixed (degree)	$O(\lvert Q_1\rvert)$
Graph attention (GAT)	1-hop	learned	$O(\lvert Q_1\rvert)$
Walk operator $W^{(g)}$	walk-reachable	fixed (multiplicity)	$O(\lvert\mathcal{S}_g\rvert)$
Graph Transformer	all pairs	learned	$O(n^2)$
Walk Attention (ours)	**walk-reachable**	**learned**	$O(\lvert\mathcal{S}_g\rvert)$

4.3 Relation to message passing and Transformers

Table 1 situates Walk Attention among related operators by two axes: the *support* of interaction and whether the weights are *learned*. A one-hop graph-attention layer [10] learns weights but only over direct neighbours, so it must route long-range signal through L stacked layers—precisely the regime where over-squashing bites. A full graph Transformer [9] learns weights over all pairs, paying $O(n^2)$ and discarding graph structure unless it is reinjected by positional encodings. The fixed-weight walk operator has the right multi-hop support but cannot select. Walk Attention occupies the remaining cell: multi-hop, structure-aware support *and* learned, selective weights.

5 Experiments

We evaluate Walk Attention on a controlled family of graphs in which over-squashing is severe and *tunable*, so that the contribution of each architectural ingredient can be isolated. All code, configurations and a full experiment-tracking log are released with the paper.

5.1 A tunable over-squashing benchmark

Bottleneck-chain graphs. We build a family parameterised by a *depth* d, a number of sources K and a width M. The graph has K source vertices, then d consecutive *bottleneck layers* of M interchangeable vertices each, and finally one target vertex. Every source connects to every vertex of the first layer, every vertex of layer r connects to every vertex of layer $r+1$, and every vertex of the last layer connects to the target. Because the bottleneck vertices are interchangeable, the number of length-$(d{+}1)$ walks from a source to the target is M^d, so the walk multiplicity into the target grows as $n_{d+1}(\cdot, \text{target}) \sim K\,M^d$ (Proposition 1); a message-passing GNN must squash all of them into the target's fixed-width vector. The walk-reachability support, by contrast, remains sparse: at the deepest range it is only the K source–target pairs, about 2% of all node pairs in our configuration.

Table 2. Target accuracy (mean ± std over 5 seeds) on bottleneck-chain retrieval, K=5, M=4. Chance is $1/K = 0.20$. Depth d is the number of bottleneck layers; the answer lies d+1 hops away.

Model	**depth** $d = 2$	**depth** $d = 3$
GAT (one-hop attention)	0.44 ± 0.12	0.26 ± 0.10
Walk operator (fixed weights)	0.62 ± 0.12	0.50 ± 0.12
Walk Attention (ours)	$\mathbf{1.000 \pm 0.000}$	$\mathbf{1.000 \pm 0.000}$

Task. Each source carries a one-hot *key* and a scalar *payload*; the target carries a query key. The model must output, at the target node, the key of the source whose key matches the query—a retrieval task whose answer sits d+1 hops away, behind the bottleneck. Accuracy is measured at the target node; chance is $1/K$.

Models. We compare:

- **GAT** [10]: one-hop graph attention, d+1 layers (learned weights, one-hop support);
- **Walk operator** (`walkraw`): the fixed-weight multi-hop operator $W^{(g)}$ of Definition 4 (multi-hop support, fixed multiplicity weights);
- **Walk Attention** (ours): Algorithm 1 (multi-hop support, learned weights).

All models use hidden width 16, four heads where applicable, and d+1 layers/ranges. We train with AdamW, a ten-epoch linear learning-rate warmup followed by cosine decay, and gradient clipping; Walk Attention additionally uses layer normalisation after each layer. We report mean and standard deviation of target accuracy over five random seeds, at problem depths $d \in \{2, 3\}$ with $K = 5$, $M = 4$.

5.2 Main result

Table 2 reports the results. One-hop GAT collapses towards chance (0.26–0.44): it cannot carry the signal across the bottleneck, as expected of a local operator under over-squashing. The fixed-weight walk operator reaches the target—its multi-hop support bypasses the bottleneck—but, weighting all reachable sources by multiplicity alone, it cannot select the matching source and plateaus at 0.50–0.62. **Walk Attention solves the task exactly**, attaining 1.000 ± 0.000 accuracy at both depths across all five seeds: the same multi-hop support, now with learned query–key weights, lets the target attend selectively to the relevant source.

On stability. Without layer normalisation and warmup, Walk Attention has a high ceiling but is high-variance: across seeds its accuracy ranged from 0.44 to 1.00. The three standard stabilisers (layer normalisation, learning-rate warmup, gradient clipping) remove this variance entirely, yielding the ± 0.000 in Table 2. The instability was therefore an optimisation artefact, not a property of the operator.

Table 3. De-duplicating walks via the quotient $\Bbbk Q/I$ *hurts*: target accuracy (mean over 3 seeds), bottleneck retrieval. This is a separate ablation (3 seeds, without the stabilisers of Table 2) that isolates the operator change, so the walk-operator column is not directly comparable to Table 2; what matters here is the within-row gap: the quotient operator is consistently below the unquotiented one.

Operator	**depth** $d = 2$	**depth** $d = 3$
Quotient $\Bbbk Q/I$ (collapsed walks)	0.47	0.52
Walk operator (raw walks)	**0.64**	**0.53**

5.3 A negative result: collapsing walks hurts

The path-algebra quotient $\Bbbk Q/I$ of Section 3 suggests an alternative remedy: replace the walk multiplicities $n_g(i, j)$ by the smaller quotient dimensions $\dim(e_j(\Bbbk Q/I)_g e_i)$, de-duplicating equivalent walks before aggregation. We implemented this (the fixed-weight operator built from the quotient dimensions) and compared it to the unquotiented walk operator on the same task, in a separate controlled ablation that changes *only* the operator (quotient vs. raw) and holds everything else fixed. As Table 3 shows, **the quotient operator underperforms the raw one**: collapsing parallel walks discards the source multiplicity that the retrieval task needs. In this regime redundant paths are *signal*, not noise, and removing them removes information.

This negative result is informative: it tells us that the value of the path algebra here is *not* compression. The right use of the algebra is to supply the multi-hop attention *support* (which it does, sparsely and exactly), and to let the network learn weights over that support—which is precisely Walk Attention. We regard finding a regime in which the quotient's compression is itself beneficial (e.g. when parallel paths are genuinely redundant noise, or when distinct path classes should be routed to distinct heads) as an open question rather than a settled negative.

5.4 Cost

Walk Attention operates on the walk-reachability support, whose size is the number of nonzero entries of A^g. On the bottleneck family this is a small fraction of n^2 (about 2% at the deepest range), so the layer is far cheaper than a dense graph Transformer while retaining its long range. The supports are precomputed once per graph topology and cached, so the path-algebra computation does not enter the training loop.

6 Conclusion and Future Work

We introduced **Walk Attention**, an architecture that mitigates over-squashing in graph neural networks by attending over a multi-hop support supplied by the path algebra of the underlying quiver. The graded path algebra gives, through

Proposition 1, an exact account of which node pairs are connected by length-g walks and with what multiplicity; we use this to define a sparse attention whose support is algebraically determined and whose weights are learned. On a tunable bottleneck benchmark, Walk Attention solved a long-range retrieval task exactly (1.000 ± 0.000 over five seeds) where a one-hop attention network collapsed to chance and a fixed-weight multi-hop aggregator only partially succeeded. A complementary negative result showed that using the path-algebra *quotient* to compress equivalent walks is counterproductive when path multiplicity carries signal; the algebra's value is to shape the attention support, not to discard information.

The construction is deliberately elementary and self-contained, so that it can serve as an entry point connecting quiver representation theory to graph representation learning. Several directions follow naturally.

- **Real-world long-range benchmarks.** Beyond the controlled bottleneck family, evaluating Walk Attention on molecular and other long-range graph benchmarks, and comparing against rewiring [8] and graph-Transformer baselines, is the immediate next step.
- **Quotient-shaped supports.** The quotient $\Bbbk Q/I$, rather than compressing the signal, can *partition* walks into equivalence classes; routing distinct classes to distinct attention heads would let the algebra structure the heads themselves. Identifying which relations I are useful, and learning them from data, is an appealing problem.
- **Scaling the support.** For dense graphs the range-g support can itself grow large; truncating, sampling, or learning a sparser admissible support are natural ways to control cost while preserving long range.

Reproducibility. All graph generators, the path-algebra walk operators, the Walk Attention layer, the baselines, and the experiment-tracking logs are released. The path-algebra computations build on an open-source quiver/path-algebra library; the present work is, to our knowledge, the first published application of that algebraic machinery to graph neural networks, and we have therefore kept the algebraic construction explicit throughout.

References

1. Alon, U., Yahav, E.: On the bottleneck of graph neural networks and its practical implications. In: International Conference on Learning Representations (ICLR) (2021)
2. Assem, I., Simson, D., Skowroński, A.: Elements of the Representation Theory of Associative Algebras: Volume 1. Cambridge University Press (2006)
3. Di Giovanni, F., Giusti, L., Barbero, F., Luise, G., Lio, P., Bronstein, M.M.: On over-squashing in message passing neural networks: The impact of width, depth, and topology. In: Proceedings of the 40th International Conference on Machine Learning (ICML). PMLR, vol. 202, pp. 7865–7885 (2023)

4. Gilmer, J., Schoenholz, S.S., Riley, P.F., Vinyals, O., Dahl, G.E.: Neural message passing for quantum chemistry. In: Proceedings of the 34th International Conference on Machine Learning (ICML). PMLR, vol. 70, pp. 1263–1272 (2017)
5. Moreno Cañadas, A., Rodríguez-Nieto, J.G., Salazar Díaz, O.P.: Brauer configuration algebras induced by integer partitions and their applications in the theory of branched coverings. Mathematics **12**(22), 3626 (2024). https://doi.org/10.3390/math12223626
6. Osorio Angarita, M.A., Moreno Cañadas, A.: Brauer configuration algebras for multimedia based cryptography and security applications. Multimedia Tools and Applications **80**(15), 23485–23510 (2021). https://doi.org/10.1007/s11042-020-10239-3
7. Schiffler, R.: Quiver Representations. Springer (2014)
8. Topping, J., Di Giovanni, F., Chamberlain, B.P., Dong, X., Bronstein, M.M.: Understanding over-squashing and bottlenecks on graphs via curvature. In: International Conference on Learning Representations (ICLR) (2022)
9. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 30 (2017)
10. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y.: Graph attention networks. arXiv preprint arXiv:1710.10903 (2017)